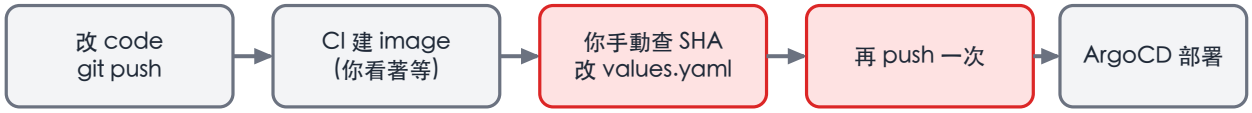
Day 22:CI 自動更新 image tag

把 GitOps 最後一個手動步驟自動化:CI 建好 image 之後,自己把新 tag 寫回 git,ArgoCD 接手部署。

今天一句話總結

以前我要「先 push 程式 → 等 CI 建 image → 手動查 SHA 改 values.yaml → 再 push」,現在只要 git push 一次,剩下的(測試、建 image、更新 tag、部署)全自動。

Day 20(之前):要手動分兩次 push



為什麼要分兩次:tag 就是 commit 的 SHA,但「改 tag」本身又是一個新 commit——自己指不到自己,
所以只能先推一次讓 CI 建 image,再回頭手動寫 tag。人一忙就會忘或寫錯。

Day 22(現在):全自動



你唯一做的事:git push。CI 只改 git,不碰 cluster;部署這件事完全交給 ArgoCD。

今天實際發生的事(時間線)

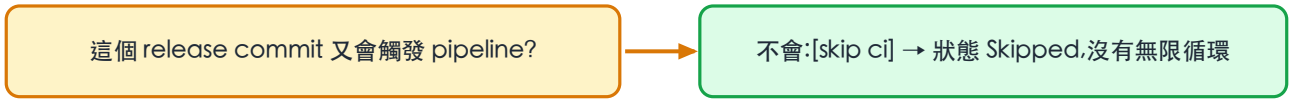
步驟	結果
在 .gitlab-ci.yml 新增第三個階段 release(bump-image-tag job)	推上去
第一版 YAML 有語法錯誤(script 裡 tag: 後面有冒號+空白),我先推了才發現	整條 pipeline 起不來,立刻修好重推
test 綠、build 綠,release 紅	log 顯示 CI_PUSH_TOKEN 是空的
補上 CI 變數 CI_PUSH_TOKEN,按 Retry	release 綠,倉庫多一筆 release … [skip ci]

release 階段到底做了什麼



任一階段失敗 → 後面的階段不會跑 → 壞版本的 tag 永遠進不了 git

1. 檢查 $CI_PUSH_TOKEN 有沒有設(沒設就明確報錯)
2. 用 token clone 一份乾淨的 main(不用 CI 預設的 detached checkout)
3. sed 把 values.yaml 裡 backend / frontend 兩行 tag 換成這次的 commit 短 SHA
4. 防呆:確認剛好改到 2 行,不是 0 行也不是 5 行
5. commit,訊息帶 [skip ci],然後 push 回 main
6. 若 push 被擋(兩條 pipeline 同時結束)→ pull --rebase 再推一次



為什麼 CI 不直接 kubectl apply?

因為 cluster 的狀態要由 git 決定。CI 只負責「改 git」,ArgoCD 負責「讓 cluster 跟 git 一致」。好處:每次部署都有 git 紀錄(誰、什麼時候、改了什麼)、可以 git revert 回滾、CI 不需要持有 cluster 的憑證(少一個要保管的金鑰)。

為什麼要 [skip ci]?

release commit 會改 main。如果它又觸發 pipeline,pipeline 又會產生新的 release commit,就變成無限循環。[skip ci] 讓這個 commit 的 pipeline 直接標成 Skipped(你截圖裡灰色那筆就是)。

為什麼測試失敗的版本不會被部署?

GitLab 的規則:前一個階段有任何 job 失敗,後面的階段不執行。release 在最後,所以壞測試或 build 失敗的 commit,它的 tag 根本不會被寫進 values.yaml,ArgoCD 也就看不到它。(Day 24 會實際演練這件事。)

今天踩的坑:一步一步追

#3017308764 commit 657cd01c	設定檔 YAML 語法錯 紅:整條 pipeline 起不來
#3017309723 commit 13070539	test 綠 / build 綠 / release 紅 紅:CI_PUSH_TOKEN 是空的
重跑 release job commit 13070539	存好變數後按 Retry 綠:三階段全過
#3020045447 commit 2613ac38	release 13070539 [skip ci] Skipped:CI 自己 push 的 commit

你的截圖其實就是在還原這個過程

第一張(兩個 Failed):#3017308764 是設定檔錯誤,我推出去時沒先驗證;#3017309723 是 release job 失敗。第二張:重跑後三階段全綠,而且出現 release 13070539 [skip ci],作者是 CI 機器人。這筆 commit 就是證據——我沒有碰過它,是 CI 自己寫進去的。

學到的除錯方法

現象	怎麼判斷
「CI/CD configuration file 有錯」	不是 job 失敗,是 YAML 本身壞了,pipeline 連 job 都沒建出來。先在本機用 YAML parser 驗證再 push
test、build 綠,最後一關紅	前面都對,問題只在新加的那一段。直接點進紅的 job 看 log 最後幾行
log 寫「缺少 CI 變數」	這是我自己加的防呆,不是 GitLab 的錯誤。好的錯誤訊息省下大量除錯時間

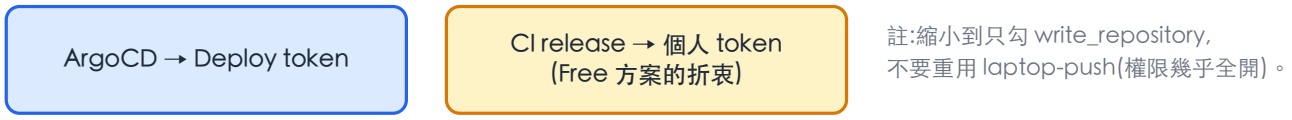
關於 YAML 的小知識

script 裡的一行如果含有「冒號+空白」(例如 tag:),YAML 會以為是 key: value,導致解析錯誤。解法:用 - | 區塊寫法,或把整行用引號包起來。

Token:為什麼這麼麻煩,以及我們的取捨



我們實際用的:



三種 token 比較

種類	權限範圍	適合做什麼	這個專案
Deploy token	專案內,只讀(repo / registry)	ArgoCD 拉程式、cluster 拉 image	已在用
Personal access token	你的帳號,scope 可調	本機 git push、CI 推送	CI 暫用(縮小到 write_repository)
Project access token	單一專案,最小權限	CI 推送(最理想)	GitLab Free 不能建

風險要講清楚

個人 token 屬於你的帳號。存進 CI 變數後,任何能改 .gitlab-ci.yml 的人都可能讓 job 把它印出來。目前專案只有你一個人,風險小;但正式環境不該這樣做。我們做了兩個緩解:勾 Masked(log 會遮住)、把 scope 縮到只有 write_repository。

更好的做法(面試可以講)

- 付費方案的 project access token(綁單一專案)
- deploy key(只能推這一個 repo 的 SSH 金鑰)
- 不讓 CI 寫 git,改成 CI 直接呼叫 ArgoCD / 用 ArgoCD Image Updater 自動偵測新 image(但那是另一套架構)

小插曲:laptop-push

你截圖裡的 laptop-push 是「Legacy token」,幾乎所有 scope 都開著。它只用來在本機推程式碼,其實只需要 write_repository。這種「權限開太大」的 token 就是面試官愛問的資安點:最小權限原則(least privilege)。

面試可以講的點(Q&A)

Q:你的 CI/CD 流程是什麼?

改 code push 到 GitLab → CI 跑 pytest 與 helm lint → arm64 runner 建 image 推 registry → release 階段把新 tag 寫回 values.yaml → ArgoCD 偵測到 git 變化自動同步到 AKS。從 push 到上線我不用做任何手動步驟。

Q:為什麼 CI 要把 tag 寫回 git,而不是直接部署?

這是 GitOps:git 是唯一真相。這樣每次部署都有 commit 紀錄、可以 revert 回滾,CI 也不需要 cluster 憑證。部署的責任交給 ArgoCD。

Q:怎麼避免 CI 自己的 commit 造成無限循環?

release commit 訊息加 [skip ci],GitLab 會跳過它的 pipeline。另外 job 裡有防呆:改到的 tag 行數不是 2 行就失敗。

Q:測試失敗會發生什麼?

stage 順序是 test → build → release,前面失敗後面不跑,所以壞版本的 tag 不會進 git,ArgoCD 不會部署它。(這是設計上的保證;Day 24 會實際演練驗證。)

Q:CI 用什麼憑證推 git?安全嗎?

用個人 access token,只給 write_repository,存成 masked CI 變數。我知道這不是最佳實務——GitLab Free 不能建專案 token——正式環境應改用專案層級 token 或 deploy key,並設短效期。

Q:遇到最難的問題是什麼?

image 架構不符:AKS 節點是 ARM,GitLab 預設 runner 是 amd64,image 上去會 exec format error。解法是指定 arm64 runner 原生 build。(Day 16 踩到)

Q:如果兩個 pipeline 同時結束,都想 push 怎麼辦?

第二個 push 會被擋(non-fast-forward)。release job 在失敗時會 pull --rebase 再推一次。這個我設計了處理,但沒有實際壓力測試過。

誠實的限制 / 還沒做的事

項目	說明
CI 用個人 token	Free 方案限制;權限已縮小,但仍綁個人帳號。README 的「已知限制」要寫
沒有人工核准關卡	main 一合併就自動上線,沒有 staging、沒有手動 approve。練習專案可接受,正式環境需要
沒有自動回滾	新版本壞了不會自動退回;目前靠 git revert + ArgoCD 同步
沒做「release 失敗」告警	如果 release job 紅了,只有看 pipeline 才知道
GitHub 不會收到 release commit	CI 只推 GitLab。本機 git pull 後要自己再推 GitHub(兩邊 remote 要同步)
並發 push 的處理沒壓測	有 rebase 重試,但沒實際讓兩條 pipeline 撞在一起驗證

接下來

Day 23:改 code → push → 看 CI 與 ArgoCD 自動上線,連續跑 3 次,每次用不同的小改動確認新版本真的在 cluster 裡。

Day 24:故意推一個會失敗的測試,證明 pipeline 擋下、壞版本不會被部署。